DSBDA Viva Theory — Comprehensive Guide

> Concise explanations with examples for all 11 practicals.

---

1. Data Wrangling I

Data Wrangling / Data Preprocessing

Raw data is messy — missing values, wrong types, inconsistent formats. Data wrangling is the process of cleaning and transforming raw data into a structured format suitable for analysis.

DataFrame & Series

- **DataFrame:** 2D tabular structure (like an Excel sheet). Rows = records, columns = features.
- **Series:** A single column of a DataFrame.

```
import pandas as pd
df = pd.read_csv("data.csv")      # DataFrame
col = df["Age"]                    # Series
```

Missing Values — `isnull()`, `isna()`

Both functions return `True` for missing (NaN) values. `isnull()` and `isna()` are identical.

```
df.isnull().sum()  # Count missing values per column
```

Handling Missing Values

Method	What it does	Example
`dropna()`	Removes rows/cols with NaN	`df.dropna()`
`fillna(value)`	Replaces NaN with a value	`df.fillna(df.mean())`

Example:

```
Age column: [25, NaN, 30, NaN, 35]
fillna(mean=30) → [25, 30, 30, 30, 35]
```

`describe()`, `info()`, `shape`

- `df.describe()` — summary statistics (count, mean, std, min, 25%, 50%, 75%, max) for numeric columns.

- `df.info()` — datatypes, non-null count, memory usage.
- `df.shape` — (rows, columns) tuple.

Datatype Checking & Conversion

```
df.dtypes           # Check types
df["col"] = df["col"].astype("float")   # Convert
```

Categorical vs Numerical Data

- **Numerical:** Numbers you can do math on (age, price, temperature).
- **Categorical:** Categories/labels (gender, color, city).

Label Encoding vs One-Hot Encoding

Encoding	What it does	Example
Label Encoding	Assigns 0, 1, 2... to categories	Red=0, Green=1, Blue=2
One-Hot Encoding	Creates binary columns for each category	Red → [1,0,0], Green → [0,1,0], Blue → [0,0,1]

Problem with label encoding: Imposes ordinal relationship (Green > Red). Use one-hot for nominal data.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])   # Male=0, Female=1

pd.get_dummies(df["Color"])   # One-hot encoding
```

Normalization vs Standardization

Method	What it does	Formula	Range
Normalization (Min-Max)	Scales to [0,1]	`(x - min) / (max - min)`	0 to 1
Standardization (Z-score)	Centers at 0, std=1	`(x - mean) / std`	No fixed range

When to use what? Standardization is preferred when data has outliers. Normalization is good when you need bounded ranges (e.g., neural networks).

---

2. Data Wrangling II

Missing Values & Inconsistent Data

Same as above — `isnull()`, `dropna()`, `fillna()`. Additionally:

- **Inconsistent data:** Same category spelled differently (e.g., "M", "Male", "male"). Fix with string cleaning: `df["Gender"].str.lower().str.strip()`.

Duplicate Data

```
df.duplicated().sum()     # Count duplicates
df.drop_duplicates()      # Remove duplicates
```

Invalid Values

Values that don't make sense (e.g., negative age, temperature of 500°C). Detect and replace/remove based on domain knowledge.

Outliers

Values far from the rest. Detect using boxplot or IQR method.

Boxplot Anatomy

```
        Lower     Q1   Median   Q3    Upper
       whisker          |           whisker
          |-----|-------|-------|-----|
                 |<--- IQR --->|
               (Q3 - Q1)
```

- **Q1 (25th percentile):** 25% of data below this.
- **Q3 (75th percentile):** 75% of data below this.
- **IQR = Q3 - Q1**
- **Lower bound = Q1 - 1.5 × IQR**
- **Upper bound = Q3 + 1.5 × IQR**
- Points outside bounds = outliers.

Outlier Removal using IQR

```
Q1 = df["Age"].quantile(0.25)
Q3 = df["Age"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df_no_outliers = df[(df["Age"] >= lower) & (df["Age"] <= upper)]
```

Skewness

Measures asymmetry of distribution:

- **Positive skew:** Tail on right (mean > median).
- **Negative skew:** Tail on left (mean < median).
- **Zero:** Symmetric (normal distribution).

Data Transformation (to fix skewness)

Method	What it does	When to use
Log transform	`log(x)`	Positive skew, large values
Square-root	`sqrt(x)`	Moderate positive skew (count data)

```
import numpy as np
df["log_col"] = np.log(df["col"])        # log transform
df["sqrt_col"] = np.sqrt(df["col"])      # sqrt transform
```

Min-Max Scaling vs Z-Score Scaling

(Already covered above — these are the same as normalization/standardization. Min-Max is sensitive to outliers; Z-score is robust.)

---

3. Descriptive Statistics

Measures of Central Tendency

Measure	What it is	Example: [10, 20, 20, 30, 80]
Mean	Sum ÷ count	(10+20+20+30+80)/5 = 32
Median	Middle value when sorted	20
Mode	Most frequent value	20

Key insight: Mean is sensitive to outliers (80 pulls it up). Median is robust.

Measures of Dispersion

Measure	What it is	Example
Range	Max - Min	80 - 10 = 70
Variance	Avg of squared deviations from mean	`Σ(x - μ)² / n`
Std Deviation	√variance	Same unit as data
Percentile	Value below which k% of data falls	25th percentile = Q1
Quartile	Divides data into 4 parts	Q1, Q2(median), Q3
IQR	Q3 - Q1	Spread of middle 50%

Grouped Statistics — `groupby()` + `agg()`

```
df.groupby("Species")["PetalLength"].agg(["mean", "median", "std"])
```

Example — Iris dataset:

```
Species        mean   median    std
setosa         1.46     1.50   0.17
versicolor     4.26     4.35   0.47
virginica      5.55     5.55   0.55
```

Categorical vs Quantitative Variable

- **Categorical (qualitative):** Categories — gender, species, color.
- **Quantitative (numerical):** Numbers — age, price, petal length.

Iris Dataset Basics

- 150 samples, 3 species (setosa, versicolor, virginica), 50 each.
- 4 features: sepal length, sepal width, petal length, petal width.
- Famous dataset by R.A. Fisher — used for classification and visualization.

---

4. Linear Regression

Supervised Learning

Model learns from **labeled data** (input → known output). Two types: regression (continuous output) and classification (discrete output).

Regression vs Classification

- **Regression:** Predict a number — price, temperature, salary.
- **Classification:** Predict a category — spam/not spam, species.

Linear Regression — Simple

Predicts a continuous value using a straight line:

Equation: `y = mx + c`

- `y` = dependent variable (what we predict)
- `x` = independent variable (feature)
- `m` = coefficient (slope) — how much y changes per unit x
- `c` = intercept (value of y when x = 0)

Multiple Linear Regression

More than one feature:

$$y = m_1x_1 + m_2x_2 + ... + m_nx_n + c$$

Example — Boston Housing: Predict price using: CRIM (crime rate), RM (rooms), AGE, DIS (distance), etc.

Correlation & Correlation Matrix

- **Correlation:** How two variables move together. Range: -1 to +1.
  - +1: Perfect positive (both increase together).
  - -1: Perfect negative (one increases, other decreases).
  - 0: No linear relationship.
- **Correlation matrix:** Table showing correlation between every pair of features.

```
df.corr()   # Pairwise correlation
```

Train-Test Split

Split data into training (to learn) and testing (to evaluate).

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Common split: 80% train, 20% test.

Prediction & Residual

- **Prediction:** `y_pred = model.predict(X_test)`
- **Residual = Actual - Predicted** (error for each point).

Evaluation Metrics

Metric	Formula	What it means
MAE	`mean(	actual - pred
MSE	`mean((actual - pred)²)`	Penalizes large errors more.
RMSE	`√MSE`	In same unit as target. Most common.
R^2 Score	`1 - SS_res / SS_tot`	Proportion of variance explained. 0 to 1. Higher is better.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

Outlier Effect on Regression

Outliers pull the regression line toward them — **reduces R^2**, increases error. Always check for outliers before fitting.

---

5. Logistic Regression

Classification & Binary Classification

- **Classification:** Predict a category.
- **Binary classification:** Only 2 classes (yes/no, spam/not spam, buy/don't buy).

Logistic Regression

Despite the name, it's used for **classification**, not regression. It predicts the **probability** that a sample belongs to a class.

Sigmoid Function

Maps any real number to a probability between 0 and 1:

$$\sigma(z) = 1 / (1 + e^{-z})$$

```
z = -∞  →  σ(z) = 0
z =  0  →  σ(z) = 0.5
z = +∞  →  σ(z) = 1
```

Threshold (Decision Boundary)

- If probability ≥ 0.5 → predict class 1.
- If probability < 0.5 → predict class 0.

Feature Scaling — StandardScaler

Logistic regression uses gradient descent. Features on different scales cause slow/unstable convergence.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)   # Use same scaler on test!
```

Confusion Matrix

```
              Actual Positive    Actual Negative
Predicted Pos       TP                 FP
Predicted Neg       FN                 TN
```

Term	Meaning
TP (True Positive)	Correctly predicted positive
TN (True Negative)	Correctly predicted negative
FP (False Positive)	Wrongly predicted positive (Type I error)
FN (False Negative)	Wrongly predicted negative (Type II error)

Evaluation Metrics

Metric	Formula	What it tells
Accuracy	`(TP + TN) / (TP + TN + FP + FN)`	Overall correctness
Error Rate	`1 - Accuracy`	Overall mistakes
Precision	`TP / (TP + FP)`	Of predicted positives, how many were correct
Recall	`TP / (TP + FN)`	Of actual positives, how many were caught
F1-Score	`2 × (P × R) / (P + R)`	Harmonic mean of precision & recall

```python
from sklearn.metrics import confusion_matrix, classification_report
cm = confusion_matrix(y_test, y_pred)
print(classification_report(y_test, y_pred))
```

Social Network Ads Dataset

- Users with age, estimated salary → predict whether they bought a product.
- Binary classification: Purchased (0 or 1).

---

6. Naive Bayes

Bayes Theorem

`P(A|B) = P(B|A) × P(A) / P(B)`

- `P(A|B)` = Posterior (probability of A given B has occurred)
- `P(A)` = Prior (initial belief about A)
- `P(B|A)` = Likelihood (probability of B if A is true)
- `P(B)` = Evidence

Naive Bayes Classifier

Applies Bayes theorem with a **naive assumption**: all features are independent given the class.

Real-world example: Spam detection.

- Prior: 20% of emails are spam.
- Likelihood: word "free" appears in 80% of spam, 10% of non-spam.
- Posterior: given "free" appears, what's the probability it's spam?

Conditional Probability

`P(A|B)` = probability of A happening given that B already happened.

Feature Independence Assumption

"We assume features don't influence each other." This is **naive** because in reality features are often correlated (petal length & petal width are related). Despite this, Naive Bayes works well in practice.

Gaussian Naive Bayes

Used when features are continuous and assumed to follow a normal distribution.

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Multiclass Classification

Naive Bayes naturally handles more than 2 classes (unlike logistic regression which is binary by default).

Iris example: Classify into setosa, versicolor, or virginica based on sepal/petal measurements.

Prior vs Posterior

- **Prior** `P(class)` — probability before seeing data.
- **Posterior** `P(class | features)` — updated probability after seeing data.

Evaluation

Same as logistic regression: confusion matrix, accuracy, precision, recall.

---

7. Text Analytics

NLP (Natural Language Processing)

Field of AI that deals with understanding and processing human language.

Corpus & Document

- **Corpus:** Collection of documents (entire dataset).

- **Document:** A single text (one tweet, one review, one article).

Token & Tokenization

Splitting text into smaller units.

```
from nltk.tokenize import word_tokenize, sent_tokenize
sent_tokenize("Hello world. I am learning NLP.")
# → ["Hello world.", "I am learning NLP."]

word_tokenize("Hello world!")
# → ["Hello", "world", "!"]
```

- **Word tokenization:** Split into words.
- **Sentence tokenization:** Split into sentences.

Stop Words

Common words (the, is, a, an, in) that carry little meaning. Removed to reduce noise.

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
filtered_words = [w for w in words if w.lower() not in stop_words]
```

Regular Expression (Regex)

Pattern matching in text.

```
import re
re.sub(r"[^a-zA-Z]", " ", text)  # Remove everything except letters
re.findall(r"\d+", text)          # Find all numbers
```

Stemming vs Lemmatization

Method	What it does	Example
Stemming	Crude chop-off to base form	"running" → "run", "flies" → "fli"
Lemmatization	Uses dictionary to find root word	"running" → "run", "flies" → "fly", "better" → "good"

Key difference: Lemmatization is more accurate (real word), stemming is faster (may give non-words).

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
ps = PorterStemmer()
lemmatizer = WordNetLemmatizer()
```

```
ps.stem("running")        # → "run"
lemmatizer.lemmatize("running", pos="v")  # → "run"
```

POS Tagging

Part-of-Speech tagging — marking each word as noun, verb, adjective, etc.

```
from nltk import pos_tag
pos_tag(word_tokenize("I am learning NLP"))
# → [("I", "PRP"), ("am", "VBP"), ("learning", "VBG"), ("NLP", "NN")]
```

Common tags: NN (noun), VB (verb), JJ (adjective), RB (adverb).

TF-IDF

Term	Meaning	Formula
TF (Term Frequency)	How often a word appears in a document	`count(word) / total words in doc`
IDF (Inverse Document Frequency)	How rare a word is across all docs	`log(total docs / docs containing word)`
TF-IDF	Product of TF and IDF	`TF × IDF`

Intuition: Words common in one doc but rare overall get high TF-IDF (they are "important" for that doc).

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

Bag of Words (BoW)

Simpler than TF-IDF — just counts word occurrences, ignores order and importance.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
X = cv.fit_transform(documents)
```

NLTK

Python library for NLP — provides tokenizers, stemmers, lemmatizers, stop words, POS taggers, and more.

---

8. Data Visualization I

Data Visualization

Representing data graphically to find patterns, trends, and outliers.

matplotlib & seaborn

- **matplotlib:** Low-level plotting library (fine control).
- **seaborn:** Built on matplotlib, high-level, prettier defaults, works well with DataFrames.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Titanic Dataset

- 891 passengers, columns: Survived, Pclass, Sex, Age, Fare, Embarked, etc.
- Goal: Find patterns related to survival.

Histogram

Shows distribution of a single numeric variable. Data is divided into **bins**.

```
sns.histplot(df["Fare"], bins=30)
```

Bins

Intervals on x-axis. Too few bins → lose detail. Too many → noisy.

KDE (Kernel Density Estimate)

Smooth version of histogram — estimates the probability density function.

```
sns.kdeplot(df["Fare"])              # KDE only
sns.histplot(df["Fare"], kde=True)   # Histogram + KDE
```

Distribution Plot (`displot`)

Combines histogram and KDE (seaborn's modern API).

Countplot

Bar chart for categorical data. Shows count per category.

```
sns.countplot(data=df, x="Survived")    # How many survived vs died
sns.countplot(data=df, x="Sex")          # Male vs Female count
```

Fare Distribution — Skewness

Titanic's Fare column is **right-skewed** — most tickets were cheap, a few were very expensive.

Missing Value Handling — Median Imputation

```
df["Age"].fillna(df["Age"].median(), inplace=True)
```

Median is preferred over mean when data is skewed or has outliers.

Categorical vs Numerical Visualization

- **Numerical:** histogram, boxplot, KDE.
- **Categorical:** countplot, barplot, pie chart.

---

9. Data Visualization II

Boxplot (Detailed)

Visualizes distribution using five-number summary:

```
Min   Q1 Median  Q3   Max
 |----|----|-----|----|
       |<--IQR-->|
```

- **Box:** Q1 to Q3 (middle 50%).
- **Line in box:** Median.
- **Whiskers:** Extend to min/max within 1.5×IQR.
- **Points beyond whiskers:** Outliers.

```
sns.boxplot(data=df, x="Sex", y="Age", hue="Survived")
```

Hue Parameter

Adds a third categorical dimension via color.

Example — `hue="Survived"`**:** Splits each box into two colored boxes — one for survivors, one for non-survivors. Lets you compare age distribution of survivors vs non-survivors within each gender.

Gender-wise Age Distribution (Titanic)

- Males and females had similar age ranges overall.
- But survival rate varied — "women and children first" policy.

Survival Analysis from Boxplot

Compare median ages of survivors vs non-survivors:

- Children (low age) had higher survival.
- Older passengers had lower survival.

Histogram vs Boxplot

Chart	Pros	Cons
Histogram	Shows shape of distribution, peaks, gaps	Hard to compare multiple groups
Boxplot	Great for comparing groups, shows outliers	Hides distribution shape (multimodal)

---

10. Data Visualization III

Iris Dataset — Feature Types

Feature	Type	Values
Sepal Length	Numeric (continuous)	4.3 - 7.9 cm
Sepal Width	Numeric (continuous)	2.0 - 4.4 cm
Petal Length	Numeric (continuous)	1.0 - 6.9 cm
Petal Width	Numeric (continuous)	0.1 - 2.5 cm
Species	Nominal (categorical)	setosa, versicolor, virginica

Numeric vs Nominal vs Ordinal

- **Numeric:** Measurable quantities (height, weight, petal length).
- **Nominal:** Categories with no order (species, color, city).
- **Ordinal:** Categories with order (education level: high school < bachelor < master).

Histogram per Feature

Plot all four features side by side to compare distributions:

```
df.hist(figsize=(10, 8))
```

Observations from Iris:

- Petal length & width are **bimodal** (two peaks) — setosa has small petals, the other two have larger.
- Sepal width is roughly **normal** (bell-shaped).
- Setosa is easily separable — the other two species overlap.

Boxplot per Feature

```
sns.boxplot(data=df, x="Species", y="PetalLength")
```

Outlier Detection from Boxplots

- Setosa petal length: very tight range (~1.0–1.9), no outliers.
- Versicolor petal length: 3.0–5.1, few outliers.
- Virginica petal length: 4.5–6.9, few outliers.

Species-wise Comparison

- **Setosa:** Small petals, relatively wide sepals.
- **Versicolor:** Medium petals and sepals.
- **Virginica:** Large petals, longest sepals.

---

12. WeatherAUS SVM

Weather Dataset

Daily weather observations from many Australian weather stations. Columns: Date, Location, MinTemp, MaxTemp, Rainfall, WindGustDir, WindGustSpeed, Humidity, Pressure, RainToday, RainTomorrow, etc.

Classification Task

Predict whether it will rain tomorrow (RainTomorrow: Yes/No) based on today's weather features.

SVM (Support Vector Machine)

A supervised classification algorithm. Finds the **best hyperplane** that separates classes with the **maximum margin**.

Key SVM Concepts

Concept	What it means
Hyperplane	The decision boundary separating classes. In 2D → line, in 3D → plane, in higher dimensions → hyperplane.
Support Vectors	The closest points from each class to the hyperplane. Only these matter for defining the boundary.
Margin	Distance between the hyperplane and the nearest support vectors. SVM maximizes this margin.

Intuition: SVM doesn't just separate the classes — it finds the boundary that gives the **widest possible gap** between them.

Kernel Trick

When data isn't linearly separable, SVM maps it to a **higher dimension** where separation becomes possible.

```
from sklearn.svm import SVC
svm_linear = SVC(kernel="linear")
svm_poly = SVC(kernel="poly", degree=3)
svm_rbf = SVC(kernel="rbf")   # Default, good for non-linear data
```

Kernel	When to use
Linear	Data is linearly separable
Polynomial	Moderate non-linearity
RBF (Radial Basis Function)	Complex non-linear boundaries (most common)

Label Encoding for Weather Data

Convert categorical columns (Location, WindGustDir, RainToday) to numbers:

```
from sklearn.preprocessing import LabelEncoder
for col in ["Location", "WindGustDir", "RainToday"]:
    df[col] = LabelEncoder().fit_transform(df[col])
```

Feature Scaling with StandardScaler

SVM is sensitive to feature scales (features with larger values dominate the distance calculation).

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Evaluation: Accuracy, Confusion Matrix, Classification Report

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Data Leakage — CRITICAL!

Data leakage happens when information from the future (or test set) leaks into the training data, making the model appear artificially good.

The `RISK_MM` trap in WeatherAUS: `RISK_MM` is the amount of rainfall **measured at the end of the day**. Since `RainTomorrow` is also about rainfall, having `RISK_MM` in features is cheating — the model sees tomorrow's rainfall to predict tomorrow's rainfall.

Rule: Always drop `RISK_MM` before training.

```
df.drop("RISK_MM", axis=1, inplace=True)
```

Other leakage examples: scaling before splitting (use `fit_transform` on train, `transform` on test).

`RainToday` vs `RainTomorrow`

- `RainToday` = Did it rain today? (Yes/No). Safe to use as feature.
- `RainTomorrow` = Target variable. What we're predicting.
- `RISK_MM` = Rainfall amount today → **do NOT use** as feature (leakage).

Must-Know Summary (Quick Reference)

Data Cleaning

Concept	Key Point
Missing values	`isnull().sum()`, handle with `dropna()` / `fillna(mean/median)`
Outliers	IQR method: Q1 - 1.5×IQR to Q3 + 1.5×IQR
Encoding	Label (ordinal) vs One-hot (nominal)
Scaling	Min-Max [0,1] vs Standardization (mean=0, std=1)

Statistics

Concept	Key Point
Mean	Sum/count. Sensitive to outliers.
Median	Middle value. Robust to outliers.
Std Dev	√variance. Spread of data.
IQR	Q3 - Q1. Spread of middle 50%.

ML Basics

Concept	Key Point
Supervised learning	Labeled data (X → y). Regression (continuous) or classification (discrete).

Concept	Key Point
Train-test split	80-20. Learn on train, evaluate on test.
Feature scaling	Essential for logistic regression & SVM.

Models

Model	Type	Key Idea
Linear Regression	Regression	`y = mx + c`. Minimizes squared error.
Logistic Regression	Binary Classification	Sigmoid → probability. Threshold 0.5.
Naive Bayes	Multiclass Classification	Bayes theorem. Assumes feature independence.
SVM	Classification	Maximize margin between classes. Kernel trick for non-linear.

Evaluation

Metric	What it measures
MAE / MSE / RMSE	Regression errors
R^2	Variance explained by model
Confusion Matrix	TP / TN / FP / FN
Accuracy	Overall correctness
Precision	Of predicted positives, how many were right
Recall	Of actual positives, how many were caught
F1	Harmonic mean of precision & recall

Text Analytics

Concept	Key Point
Tokenization	Word-level, sentence-level
Stop words	Remove common words (the, is, a)
Stemming	Crude chop: "running" → "run"
Lemmatization	Dictionary-based: "better" → "good"
POS tagging	Noun, verb, adjective...
TF-IDF	Word importance in a document

Visualization

Chart	Use for
Histogram	Distribution of numeric variable
Boxplot	Compare groups, detect outliers
Countplot	Count per category
Heatmap	Correlation matrix
Scatterplot	Relationship between two numeric vars

Datasets

Dataset	Used For	Key Columns
Iris	Classification, visualization	Sepal/Petal length/width, Species
Titanic	Visualization, classification	Survived, Sex, Age, Fare, Pclass
Boston Housing	Linear Regression	RM, AGE, LSTAT, MEDV (price)
Social Network Ads	Logistic Regression	Age, Salary, Purchased
WeatherAUS	SVM	MinTemp, Humidity, RainToday, **RISK_MM** ↓